

LAPORAN PRAKTIKUM PEMROGRAMAN MOBILE C
MODUL 3 EVENT, STATE, IMPERATIF-DEKLARATIF, DAN
IMPLISIT INTENT



Dosen Pengampu:

Muhammad Aminul Akbar, S.Kom., M.T

Disusun Oleh:

Ghefira Addien M. M - 245150400111033

FAKULTAS ILMU KOMPUTER
PROGRAM STUDI SISTEM INFORMASI
UNIVERSITAS BRAWIJAYA 2026

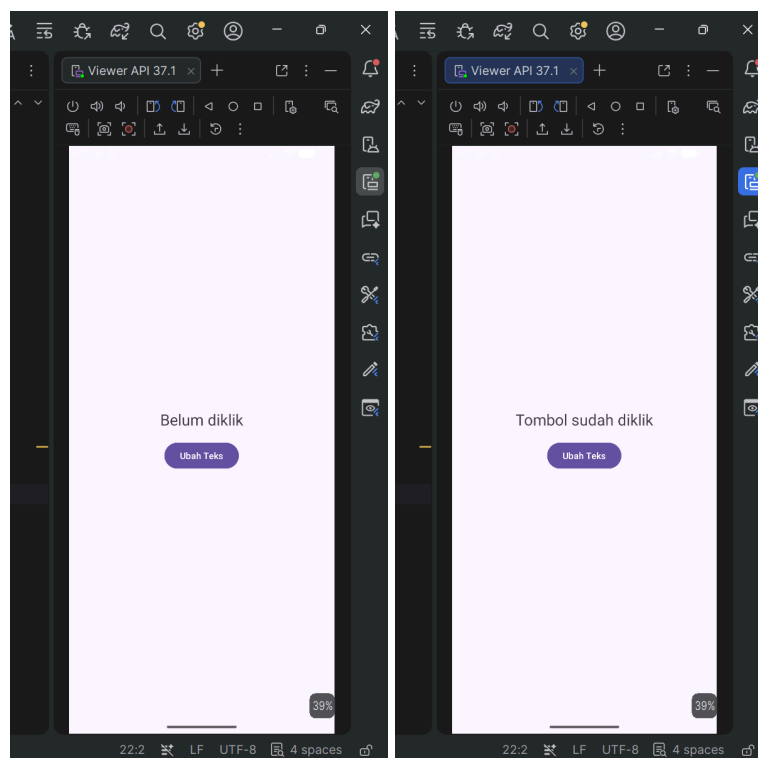
1. Latihan Praktik 1 - Perbandingan Imperatif dan Deklaratif

1.1 Hasil Praktik Project A - XML/View (Imperatif)

Pada project pertama dibuat antarmuka berbasis XML/View dengan teks awal "Belum diklik" dan tombol "Ubah Teks". Ketika tombol ditekan, program memperoleh referensi TextView melalui findViewById() kemudian mengubah properti text secara langsung. Pendekatan ini menunjukkan cara kerja imperatif karena kode secara eksplisit menentukan komponen UI yang harus diubah.

```
val tvMessage = findViewById<TextView>(R.id.tvMessage)
val btnChange = findViewById<Button>(R.id.btnChange)

btnChange.setOnClickListener {
    tvMessage.text = "Tombol sudah diklik"
}
```



Gambar 1. Kondisi awal dan setelah tombol diklik pada Project XML/View

1.2 Hasil Praktik Project B - Jetpack Compose (Deklaratif)

Pada project kedua dibuat hasil tampilan yang sama menggunakan Jetpack Compose. Teks yang tampil tidak diubah secara langsung, tetapi berasal dari state message. Event onClick mengubah nilai state tersebut. Setelah state berubah, Compose melakukan recomposition pada bagian UI yang bergantung pada message sehingga teks pada layar ikut berubah.

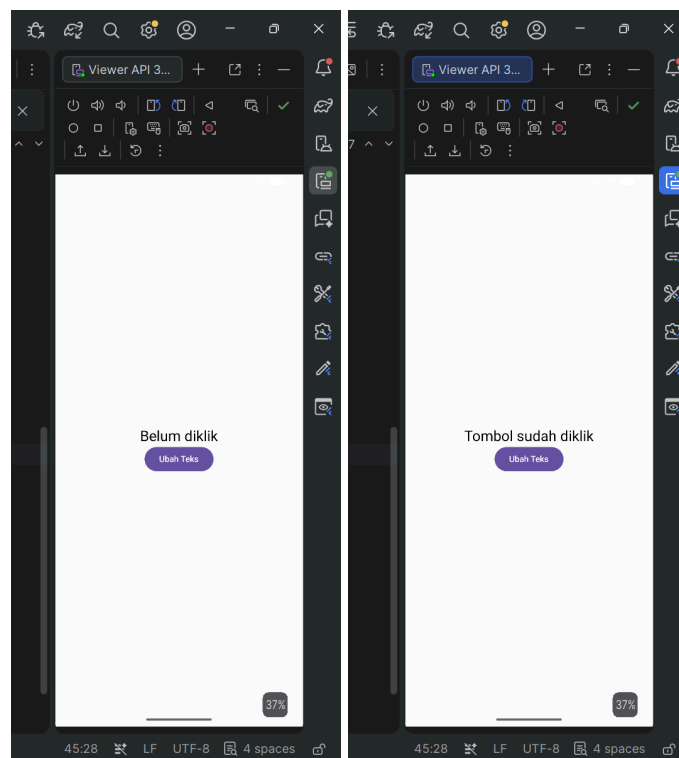
```
@Composable
fun ChangeTextScreen() {
    var message by remember {
```

```

        mutableStateOf("Belum diklik")
    }

    Column(
        modifier = Modifier.fillMaxSize(),
        horizontalAlignment = Alignment.CenterHorizontally,
        verticalArrangement = Arrangement.Center
    ) {
        Text(text = message, fontSize = 24.sp)
        Button(onClick = {
            message = "Tombol sudah diklik"
        }) {
            Text("Ubah Teks")
        }
    }
}

```



Gambar 2. Kondisi awal dan akhir setelah di klik Project Jetpack Compose

1.3 Jawaban Pertanyaan

2. Pada project XML/View, objek apa yang diubah secara langsung ketika tombol diklik?

Objek yang diubah secara langsung adalah TextView dengan referensi tvMessage. Perubahan dilakukan pada baris `tvMessage.text = "Tombol sudah diklik"`. Baris tersebut mengubah properti text milik objek View secara langsung ketika event klik terjadi.

3. Pada project Compose, nilai apa yang diubah ketika tombol diklik?

Nilai yang diubah adalah state message. Baris yang menjalankan perubahan adalah `message = "Tombol sudah diklik"` di dalam `onClick` Button. Komponen Text hanya membaca nilai dari state tersebut melalui `Text(text = message)`.

4. Mengapa pada Compose tidak terdapat perintah seperti `tvMessage.text` atau `setText()`?

Jetpack Compose menggunakan pendekatan deklaratif. Program tidak perlu mencari objek Text kemudian mengubah propertinya secara langsung. UI dideklarasikan berdasarkan state saat ini, sehingga ketika state berubah Compose akan menjalankan recomposition dan memperbarui bagian antarmuka yang terkait.

5. Jelaskan alur perubahan pada XML/View dengan pola event - View - perubahan tampilan.

Ketika tombol ditekan, event `setOnClickListener` dijalankan. Program kemudian mengakses objek `TextView` melalui referensi `tvMessage` dan mengubah properti `text`. Akibatnya, teks pada layar berubah. Alurnya adalah event klik - objek View (`TextView`) - properti `text` diubah - tampilan berubah.

6. Jelaskan alur perubahan pada Compose dengan pola event - state - recomposition - perubahan tampilan.

Ketika tombol ditekan, event `onClick` mengubah nilai state `message`. Perubahan pada `mutableStateOf` terdeteksi oleh Compose dan memicu recomposition. Fungsi `Text` kemudian dievaluasi kembali dengan nilai `message` terbaru sehingga tampilan berubah. Alurnya adalah event klik - state berubah - recomposition - UI menampilkan state terbaru.

7. Apa yang terjadi jika message diganti menjadi variabel biasa `var message = "Belum diklik"`?

Nilai variabel dapat berubah di dalam event, tetapi perubahan tersebut tidak otomatis meminta Compose memperbarui UI karena variabel biasa bukan state yang dapat diamati. Akibatnya teks pada layar tidak mengikuti perubahan seperti ketika menggunakan `mutableStateOf`. Jika composable direkomposisi oleh sebab lain, variabel lokal juga akan dibuat kembali dengan nilai awal.

8. Tuliskan kesimpulan perbedaan cara berpikir imperatif dan deklaratif.

Pada pendekatan imperatif, program menentukan komponen UI mana yang harus dicari dan diubah secara langsung. Pada pendekatan deklaratif, program mendeskripsikan seperti apa UI berdasarkan state saat ini. Event cukup mengubah state, sedangkan Compose menangani pembaruan tampilan melalui recomposition.

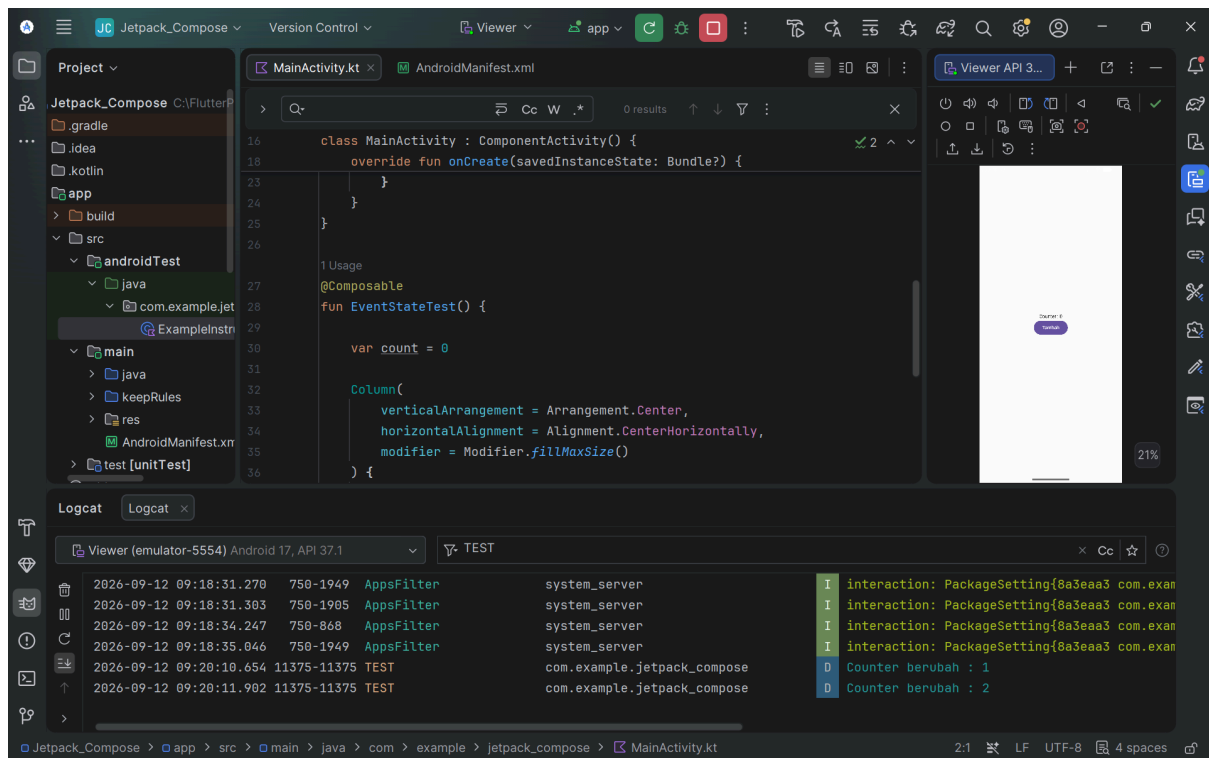
2. Latihan Praktik 2 - Event Handling State

2.1 Percobaan Menggunakan Variabel Biasa

Percobaan awal menggunakan variabel lokal count dengan nilai awal 0. Event onClick menambah count dan menulis nilai terbaru ke Logcat dengan tag TEST. Karena count masih berupa variabel biasa, perubahan nilai tersebut tidak menjadi observable state bagi Compose.

```
@Composable
fun EventStateTest() {
    var count = 0

    Column(
        verticalArrangement = Arrangement.Center,
        horizontalAlignment = Alignment.CenterHorizontally,
        modifier = Modifier.fillMaxSize()
    ) {
        Text("Counter : $count")
        Button(onClick = {
            count++
            Log.d("TEST", "Counter berubah : $count")
        }) {
            Text("Tambah")
        }
    }
}
```



Gambar 3. Kondisi Logcat setelah di klik Tambah

2. Apa yang muncul di Logcat setiap kali tombol "Tambah" diklik?

Logcat menampilkan pesan dengan tag TEST, misalnya "Counter berubah : 1", kemudian meningkat menjadi 2, 3, dan seterusnya selama composible tidak mengalami recomposition yang membuat variabel lokal dibuat ulang.

3. Apa yang terjadi pada tampilan aplikasi setiap kali tombol "Tambah" diklik?

Teks Counter pada UI tetap menampilkan nilai awal karena perubahan count tidak memicu recomposition. Walaupun nilai pada event handler berubah dan dapat terlihat di Logcat, Compose tidak mengetahui bahwa data tersebut harus digunakan untuk menggambar ulang UI.

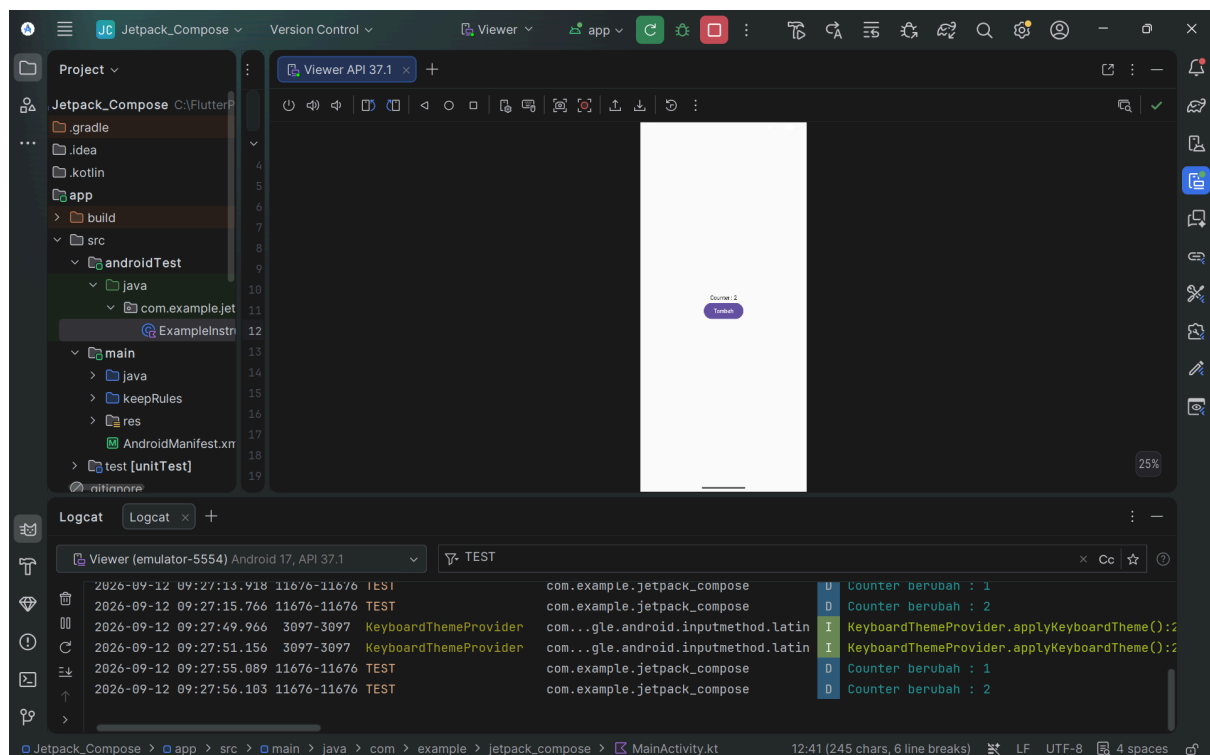
4. Apa yang menyebabkan hal tersebut terjadi?

count dideklarasikan sebagai variabel biasa, bukan state yang dapat diamati oleh Compose. Karena tidak menggunakan mutableStateOf, perubahan nilai count tidak mengirimkan sinyal agar bagian UI yang membacanya direkomposisi.

2.2 Percobaan Menggunakan remember dan mutableStateOf

Pada tahap berikutnya count diubah menjadi state menggunakan remember dan mutableStateOf. Property delegation by digunakan agar nilai state dapat dibaca dan diubah langsung tanpa menuliskan .value.

```
var count by remember { mutableStateOf(0) }
```



Gambar 4. Kondisi Logcat dan Counter setelah tombol Tambah diklik

9. Apa yang muncul di Logcat setiap kali tombol "Tambah" diklik setelah menggunakan state?

Logcat tetap menampilkan perubahan nilai counter, misalnya "Counter berubah : 1", "Counter berubah : 2", dan seterusnya. Perbedaanannya adalah nilai yang sama sekarang juga tercermin pada UI.

10. Apa yang terjadi pada tampilan aplikasi setiap kali tombol "Tambah" diklik?

Nilai Counter pada layar bertambah mengikuti jumlah klik. Setiap perubahan state count membuat Compose memperbarui bagian UI yang membaca nilai tersebut.

11. Apa yang menyebabkan hal tersebut terjadi dan berbeda dengan percobaan sebelumnya?

mutableStateOf membuat count menjadi state yang dapat diamati. Ketika count berubah, Compose menjadwalkan recomposition. remember mempertahankan nilai state selama composable masih berada dalam composition, sehingga nilai tidak kembali ke nol pada setiap recomposition.

12. Apa yang dimaksud recomposition pada Compose?

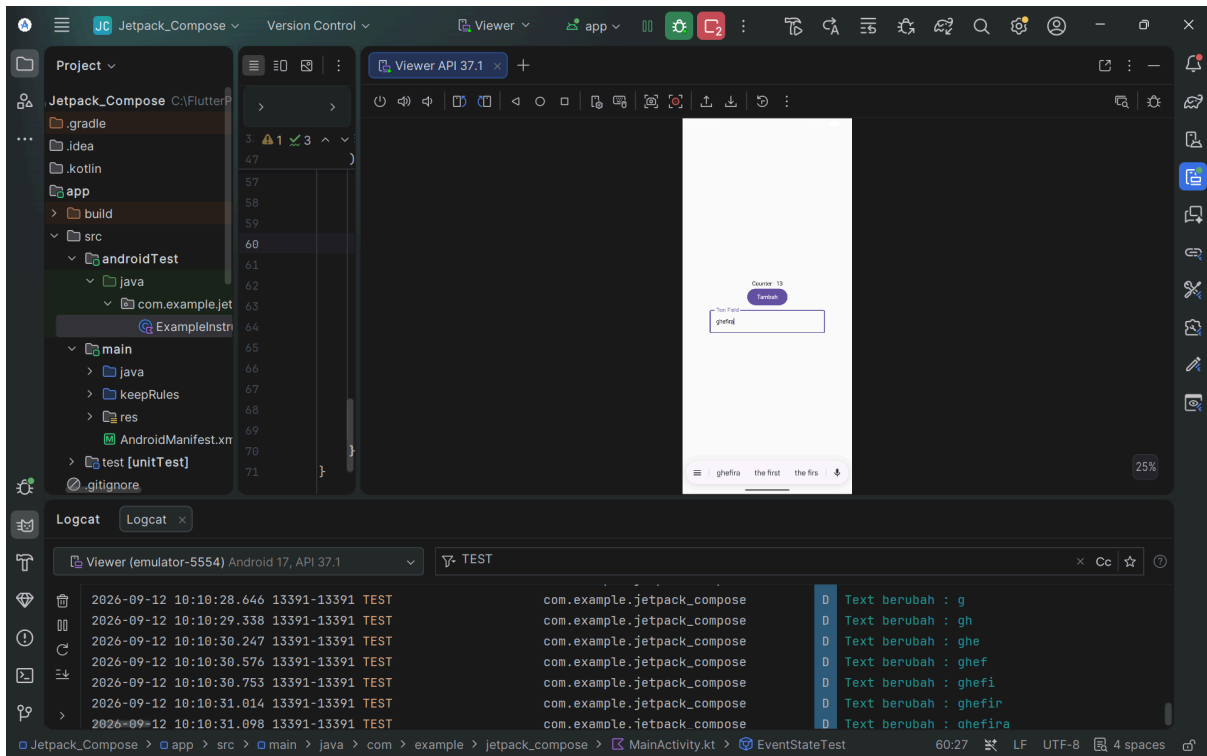
Recomposition adalah proses ketika Compose menjalankan kembali bagian composable yang bergantung pada state yang berubah, kemudian memperbarui UI yang diperlukan agar mencerminkan state terbaru.

2.3 Percobaan OutlinedTextField dan onValueChange

Percobaan terakhir menambahkan state text dan OutlinedTextField. Event onValueChange dijalankan setiap kali isi field berubah. Nilai terbaru disimpan ke state text dan juga dituliskan ke Logcat.

```
var count by remember { mutableStateOf(0) }
var text by remember { mutableStateOf("") }

OutlinedTextField(
    value = text,
    onValueChange = {
        text = it
        Log.d("TEST", "Text berubah : $it")
    },
    label = { Text("Text Field") }
)
```



Gambar 5. Kondisi Logcat dan OutlinedTextField setelah mengetik "Ghefira"

15. Apa yang muncul di Logcat setiap kali menginputkan huruf di Text Field?

Setiap perubahan input menghasilkan pesan "Text berubah : ..." dengan nilai terbaru dari field. Nilai itu pada `onValueChange` berisi keseluruhan teks setelah perubahan, sehingga Logcat akan menampilkan isi field yang terus bertambah atau berubah.

16. Apa yang terjadi pada tampilan aplikasi setiap kali menginputkan huruf di Text Field?

Karakter yang diketik langsung muncul pada `OutlinedTextField`. Hal ini terjadi karena value komponen berasal dari state text, dan `onValueChange` memperbarui state tersebut setiap kali input berubah.

17. Apa yang menyebabkan hal tersebut terjadi?

State text dibuat menggunakan `remember` dan `mutableStateOf`. Ketika `onValueChange` mengubah text, Compose melakukan recomposition sehingga `OutlinedTextField` menerima value terbaru dan menampilkannya.

18. Sebutkan state apa saja yang ada pada program yang terakhir dibuat?

Terdapat dua state, yaitu `count` bertipe `Int` yang menyimpan nilai counter dan `text` bertipe `String` yang menyimpan isi `OutlinedTextField`.

19. Sebutkan event apa saja yang ada pada program yang terakhir dibuat?

Terdapat event `onClick` pada `Button Tambah` dan event `onValueChange` pada `OutlinedTextField`.

20. Kapan event-event tersebut dijalankan?

onClick dijalankan ketika pengguna menekan tombol Tambah. onChange dijalankan setiap kali pengguna mengubah isi Text Field, misalnya dengan mengetik atau menghapus karakter.

3. Latihan Praktik 3 - Implicit Intent

3.1 Implementasi

Pada latihan implicit intent dibuat empat tombol untuk meminta sistem Android menjalankan aksi menggunakan aplikasi lain yang sesuai. Aksi yang diuji adalah membuka browser, membuka dial telepon, memilih aplikasi email, dan membuka aplikasi peta.

```
// Buka Browser
val intent = Intent(Intent.ACTION_VIEW).apply {
    data = Uri.parse("https://www.google.com")
}
context.startActivity(intent)

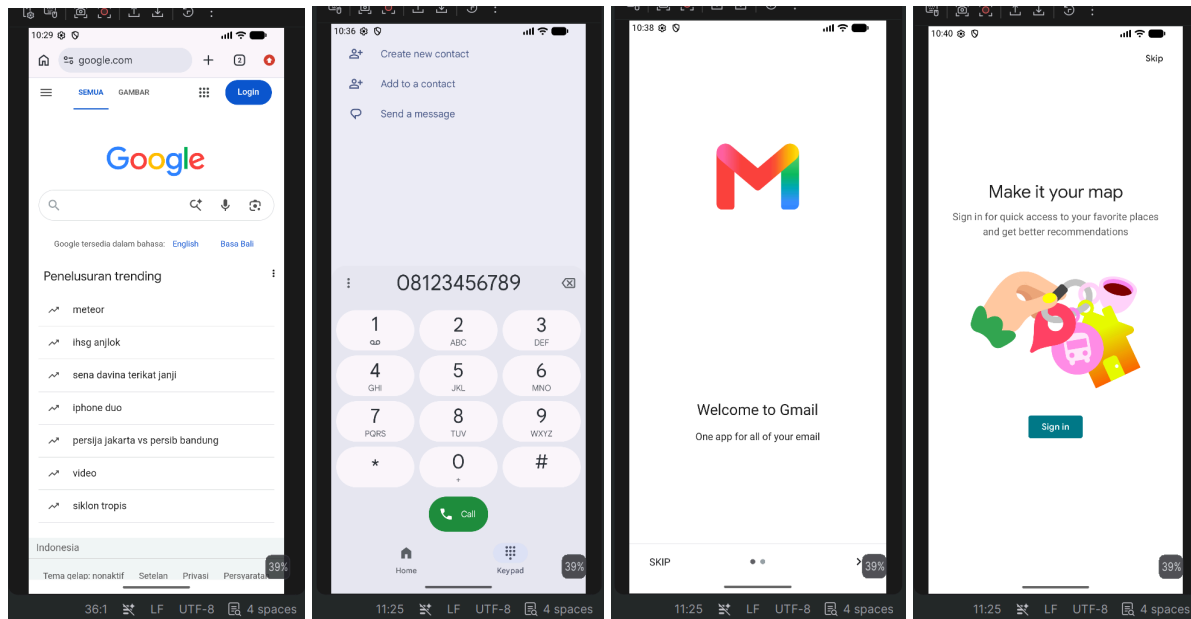
// Buka Dial Telepon
val dial = Intent(Intent.ACTION_DIAL).apply {
    data = Uri.parse("tel:08123456789")
}
context.startActivity(dial)

// Kirim Email
val email = Intent(Intent.ACTION_SENDTO).apply {
    data = Uri.parse("mailto:test@gmail.com")
}
context.startActivity(Intent.createChooser(email, "Pilih aplikasi email"))

// Buka Maps
val maps = Intent(Intent.ACTION_VIEW).apply {
    data = Uri.parse("geo:-7.250445,112.768845")
}
context.startActivity(maps)
```

3.2 Hasil Pengujian

Pengujian dilakukan terhadap empat implicit intent. Tombol “Buka Browser” berhasil membuka halaman Google melalui aplikasi browser. Tombol “Buka Telepon” membuka aplikasi dialer dengan nomor 08123456789 yang telah terisi. Tombol “Kirim Email” membuka aplikasi email yang tersedia pada emulator, sedangkan tombol “Buka Maps” membuka aplikasi peta. Aplikasi yang ditampilkan dapat berbeda bergantung pada aplikasi yang tersedia pada perangkat.



Gambar 6. Hasil pengujian Implicit Intent buka browser, dial, email, peta

3.3 Jawaban Pertanyaan

11. Apa perbedaan fungsi `Intent.ACTION_VIEW`, `Intent.ACTION_DIAL`, dan `Intent.ACTION_SENDTO`?

`Intent.ACTION_VIEW` digunakan untuk meminta sistem menampilkan atau membuka data tertentu, misalnya URL web atau lokasi geo. `Intent.ACTION_DIAL` digunakan untuk membuka aplikasi telepon/dialer dengan nomor yang telah disiapkan tanpa langsung melakukan panggilan. `Intent.ACTION_SENDTO` digunakan untuk meminta aplikasi yang mampu mengirim data ke alamat tertentu, pada percobaan ini alamat email melalui skema mailto.

16. Apa fungsi data pada intent?

data memberi informasi mengenai objek atau target data yang akan diproses oleh action. Contohnya `https://` menunjukkan halaman web, `tel:` menunjukkan nomor telepon, `mailto:` menunjukkan alamat email, dan `geo:` menunjukkan lokasi geografis.

17. Apa hubungan Action dan Data pada pilihan aplikasi yang dimunculkan sistem Android?

Sistem Android mencocokkan kombinasi action dan data dengan intent-filter aplikasi yang terpasang. Hanya aplikasi yang mampu menangani jenis aksi dan skema data tersebut yang dapat dipilih untuk menjalankan implicit intent. Karena itu action menjelaskan apa yang ingin dilakukan, sedangkan data menjelaskan terhadap informasi apa aksi tersebut dilakukan.

4. Tugas Praktik - Aplikasi Pemesanan Tiket

4.1 Implementasi Aplikasi

Pada tugas praktikum dibuat halaman minimal Pemesanan Tiket menggunakan Jetpack Compose. Harga tiket ditetapkan sebesar Rp25.000 per tiket. Jumlah tiket dapat diubah menggunakan tombol minus dan plus. Total pembayaran dihitung dari harga tiket dikalikan jumlah tiket. Tombol RESET mengembalikan jumlah tiket ke kondisi awal, yaitu satu tiket.

State utama pada aplikasi adalah jumlahTiket. State dibuat menggunakan remember dan mutableStateOf sehingga setiap perubahan jumlah tiket dapat memicu recomposition. TotalBayar tidak perlu menjadi state terpisah karena nilainya dapat diturunkan langsung dari hargaTiket dan jumlahTiket.

```
val hargaTiket = 25000
var jumlahTiket by remember { mutableStateOf(1) }
val totalBayar = hargaTiket * jumlahTiket
```

4.2 Kode Program MainActivity.kt

```
package com.example.jetpack_compose

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.foundation.layout.*
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import java.text.NumberFormat
import java.util.Locale

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            MaterialTheme {
                PemesananTiketScreen()
            }
        }
    }
}

@Composable
fun PemesananTiketScreen() {
    val hargaTiket = 25000
```

```

var jumlahTiket by remember { mutableStateOf(1) }
val totalBayar = hargaTiket * jumlahTiket

val formatRupiah =
    NumberFormat.getCurrencyInstance(Locale("id", "ID")).apply {
        maximumFractionDigits = 0
    }

Column(
    modifier = Modifier
        .fillMaxSize()
        .padding(24.dp),
    horizontalAlignment = Alignment.CenterHorizontally,
    verticalArrangement = Arrangement.Center
) {
    Text("Pemesanan Tiket", fontSize = 26.sp, fontWeight = FontWeight.Bold)
    Text("Pesan tiket dengan mudah!")
    Spacer(modifier = Modifier.height(24.dp))

    Card(modifier = Modifier.fillMaxWidth()) {
        Column(modifier = Modifier.padding(16.dp)) {
            Text("Harga Tiket", fontWeight = FontWeight.Bold)
            Text(
                formatRupiah.format(hargaTiket),
                fontSize = 26.sp,
                fontWeight = FontWeight.Bold
            )
            Text("per tiket")
        }
    }

    Spacer(modifier = Modifier.height(16.dp))

    Card(modifier = Modifier.fillMaxWidth()) {
        Column(modifier = Modifier.padding(16.dp)) {
            Text("Jumlah Tiket", fontWeight = FontWeight.Bold)
            Spacer(modifier = Modifier.height(12.dp))
            Row(
                modifier = Modifier.fillMaxWidth(),
                horizontalArrangement = Arrangement.Center,
                verticalAlignment = Alignment.CenterVertically
            ) {
                Button(
                    onClick = {
                        if (jumlahTiket > 1) jumlahTiket--
                    },
                    enabled = jumlahTiket > 1
                ) {
                    Text("-", fontSize = 24.sp)
                }
            }
        }
    }
}

```

```

        Spacer(modifier = Modifier.width(30.dp))
        Text(
            "$jumlahTiket",
            fontSize = 30.sp,
            fontWeight = FontWeight.Bold
        )
        Spacer(modifier = Modifier.width(30.dp))

        Button(onClick = { jumlahTiket++ }) {
            Text("+", fontSize = 24.sp)
        }
    }
}

Spacer(modifier = Modifier.height(16.dp))

Card(modifier = Modifier.fillMaxWidth()) {
    Column(modifier = Modifier.padding(16.dp)) {
        Text("Total", fontWeight = FontWeight.Bold)
        Text(
            formatRupiah.format(totalBayar),
            fontSize = 28.sp,
            fontWeight = FontWeight.Bold
        )
    }
}

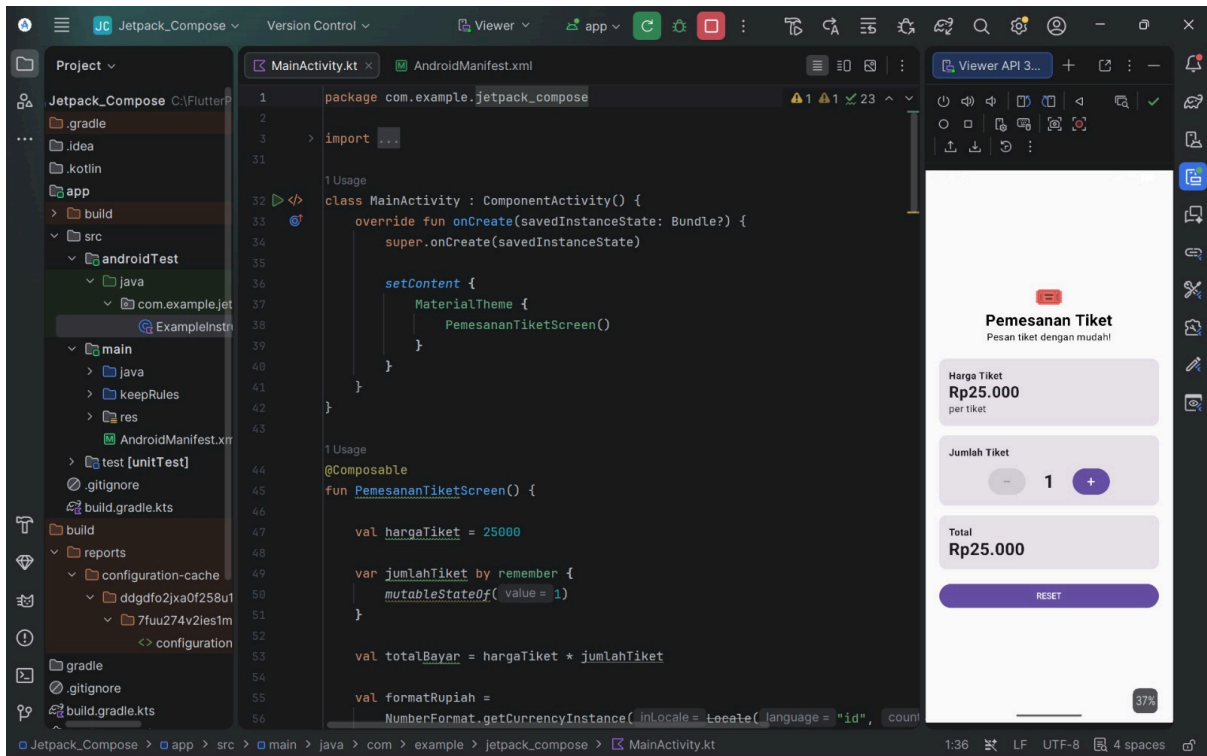
Spacer(modifier = Modifier.height(20.dp))

Button(
    modifier = Modifier.fillMaxWidth(),
    onClick = { jumlahTiket = 1 }
) {
    Text("RESET")
}
}
}

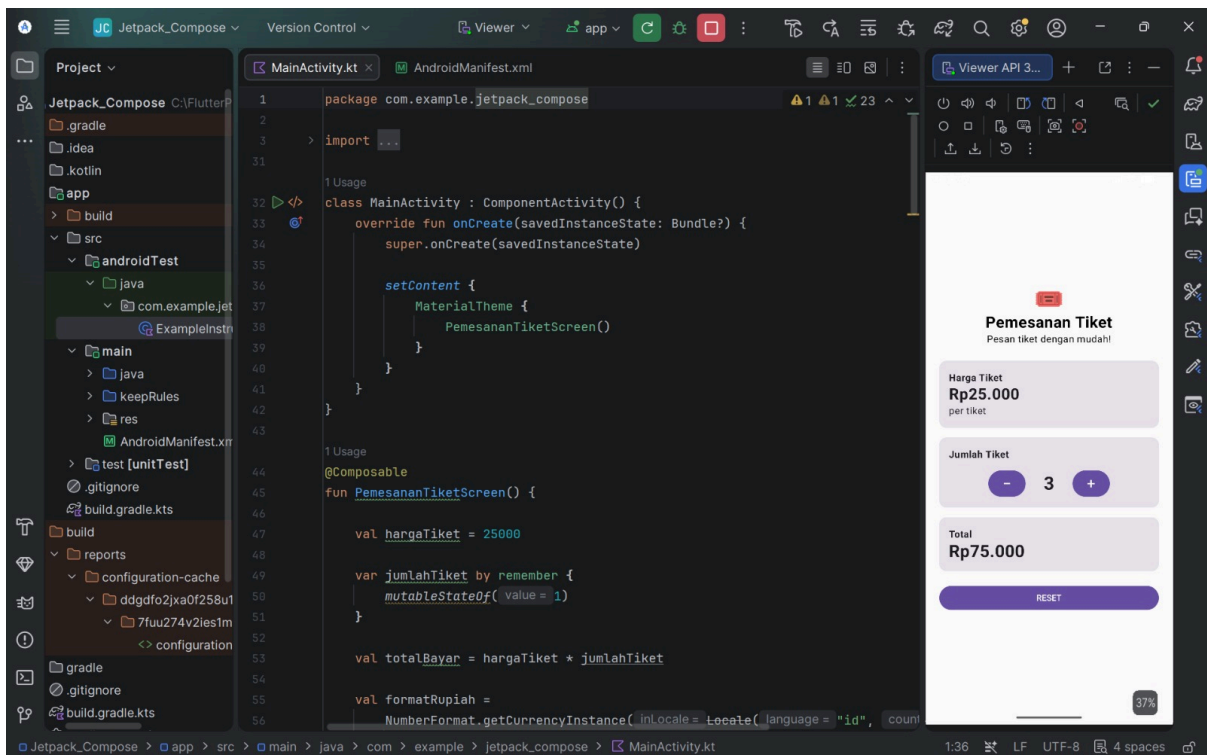
```

4.3 Hasil Pengujian

Pada kondisi awal, jumlah tiket adalah 1. Dengan harga satu tiket Rp25.000, total pembayaran yang ditampilkan adalah Rp25.000. Setelah tombol plus ditekan dua kali, jumlah tiket menjadi 3 dan total pembayaran menjadi Rp75.000. Tombol minus mengurangi jumlah tiket dengan batas minimum satu tiket, sedangkan tombol RESET mengembalikan nilai jumlahTiket menjadi 1.



Gambar 7. Kondisi awal aplikasi Pemesanan Tiket



Gambar 8. Kondisi setelah tombol plus ditekan dua kali

4.4 Analisis Event dan State

jumlahTiket merupakan state karena nilainya merepresentasikan kondisi UI saat ini dan perubahan nilainya harus terlihat pada tampilan. Event utama berasal dari onClick pada tombol plus, minus, dan RESET. Tombol plus menjalankan jumlahTiket++, tombol minus

menjalankan jumlahTiket-- selama nilainya lebih dari satu, sedangkan RESET menetapkan jumlahTiket = 1.

Ketika salah satu event tersebut mengubah jumlahTiket, Compose mendeteksi perubahan mutableStateOf lalu melakukan recomposition. Nilai totalBayar dihitung kembali menggunakan hargaTiket * jumlahTiket. Karena Text untuk jumlah tiket dan total membaca nilai terbaru, UI otomatis diperbarui. Alur ini menunjukkan pola deklaratif: event - state - recomposition - perubahan tampilan.

5. Kesimpulan

Berdasarkan praktikum Modul 3, dapat dipahami bahwa Jetpack Compose menggunakan pendekatan deklaratif dalam membangun dan memperbarui antarmuka. Event seperti `onClick` dan `onValueChange` digunakan untuk menangani interaksi pengguna, sedangkan data yang memengaruhi tampilan dikelola sebagai state. Penggunaan `remember` dan `mutableStateOf` membuat perubahan nilai state dapat memicu recomposition sehingga UI menampilkan kondisi terbaru secara otomatis.

Percobaan XML/View dan Jetpack Compose menunjukkan perbedaan cara berpikir imperatif dan deklaratif. Pada pendekatan imperatif, program mencari dan mengubah properti View secara langsung. Pada Compose, program mengubah state dan framework memperbarui UI yang bergantung pada state tersebut. Percobaan implicit intent juga menunjukkan bahwa Android dapat memilih aplikasi lain berdasarkan kombinasi action dan data yang diberikan.

Pada tugas Pemesanan Tiket, konsep event dan state diterapkan melalui tombol plus, minus, dan RESET yang mengubah state `jumlahTiket`. Perubahan tersebut menyebabkan jumlah tiket dan total pembayaran diperbarui melalui recomposition. Dengan demikian, tugas praktikum menggabungkan pemahaman mengenai event handling, state-driven UI, dan pola deklaratif pada Jetpack Compose.